```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder,OneHotEncoder,MinMaxScaler,StandardScaler
```

```
df= pd.read_csv('/content/employee_productivity_dataset.csv')
print(df)
```

```
    Employee_ID         Name Department   Age  Gender Work_Mode   Join_Date  \
0          E001  Amit Sharma         IT  25.0    Male    Remote  15-01-2021
1          E002   Riya Patel         HR  30.0  Female    Hybrid  15-02-2020
2          E003  John Mathew    Finance   NaN    Male    Onsite  10-03-2022
3          E004  Sneha Joshi         IT  28.0  Female    Remote  22-04-2021
4          E005  Arjun Verma  Marketing  35.0    Male    Hybrid  18-05-2020
..          ...          ...        ...   ...     ...       ...         ...
245        E006   Neha Singh         HR  27.0  Female    Remote  01-06-2023
246        E007    Rahul Das    Finance  40.0    Male    Onsite  07-07-2019
247        E008  Pooja Mehta  Marketing  29.0  Female    Hybrid  12-08-2022
248        E009  Sameer Khan         IT  31.0    Male    Remote  30-09-2021
249        E010   Kiran Nair         HR  26.0  Female    Onsite  05-10-2023

     Hours_Worked_Per_Week  Projects_Completed  Performance_Score   Salary  \
0                     40.0                   5                4.5  50000.0
1                     38.0                   4                4.0  45000.0
2                     45.0                   6                NaN  60000.0
3                      NaN                   5                4.2  52000.0
4                     50.0                   7                4.8      NaN
..                     ...                 ...                ...      ...
245                   36.0                   3                3.9  42000.0
246                   48.0                   8                4.7  65000.0
247                   42.0                   6                4.3  48000.0
248                    NaN                   5                4.1  51000.0
249                   37.0                   4                NaN  43000.0

      Location
0         Pune
1       Mumbai
2    Bangalore
3        Delhi
4         Pune
..         ...
245     Mumbai
246      Delhi
247  Bangalore
248       Pune
249     Mumbai

[250 rows x 12 columns]
```

Assignment Questions

Q1. Handle Missing Values (Basic Cleaning)

Fill missing values using the following:

Age → Median

Salary → Mean

Hours_Worked_Per_Week → Median

Performance_Score → Mean

Display the dataset after handling missing values

```
age_median= df['Age'].median()
salary_mean= df['Salary'].mean()
hours_median= df['Hours_Worked_Per_Week'].median()
performance_mean= df['Performance_Score'].mean()

df['Age']=df['Age'].fillna(age_median)
df['Salary']=df['Salary'].fillna(salary_mean)
df['Hours_Worked_Per_Week']=df['Hours_Worked_Per_Week'].fillna(hours_median)
df['Performance_Score']=df['Performance_Score'].fillna(performance_mean)

print(df)
```

```
    Employee_ID         Name Department   Age  Gender Work_Mode   Join_Date  \
0          E001  Amit Sharma         IT  25.0    Male    Remote  15-01-2021
1          E002   Riya Patel         HR  30.0  Female    Hybrid  15-02-2020
2          E003  John Mathew    Finance  29.0    Male    Onsite  10-03-2022
3          E004  Sneha Joshi         IT  28.0  Female    Remote  22-04-2021
4          E005  Arjun Verma  Marketing  35.0    Male    Hybrid  18-05-2020
```

```
..          ...          ...         ...   ...     ...       ...         ...
245        E006   Neha Singh          HR  27.0  Female    Remote  01-06-2023
246        E007    Rahul Das     Finance  40.0    Male    Onsite  07-07-2019
247        E008  Pooja Mehta   Marketing  29.0  Female    Hybrid  12-08-2022
248        E009  Sameer Khan          IT  31.0    Male    Remote  30-09-2021
249        E010   Kiran Nair          HR  26.0  Female    Onsite  05-10-2023

     Hours_Worked_Per_Week  Projects_Completed  Performance_Score  \
0                     40.0                   5             4.5000
1                     38.0                   4             4.0000
2                     45.0                   6             4.3125
3                     41.0                   5             4.2000
4                     50.0                   7             4.8000
..                     ...                 ...                ...
245                   36.0                   3             3.9000
246                   48.0                   8             4.7000
247                   42.0                   6             4.3000
248                   41.0                   5             4.1000
249                   37.0                   4             4.3125

           Salary   Location
0    50000.000000       Pune
1    45000.000000     Mumbai
2    60000.000000  Bangalore
3    52000.000000      Delhi
4    50666.666667       Pune
..            ...        ...
245  42000.000000     Mumbai
246  65000.000000      Delhi
247  48000.000000  Bangalore
248  51000.000000       Pune
249  43000.000000     Mumbai

[250 rows x 12 columns]
```

Q2. Label Encoding

Convert these columns into numeric values using Label Encoding:

Gender, Department

Show the updated columns.

```
scaler= LabelEncoder()
df['Gender_encoded']= scaler.fit_transform(df['Gender'])
df['Department_encoded']= scaler.fit_transform(df['Department'])

print(df)
```

```
    Employee_ID         Name  Department   Age  Gender Work_Mode   Join_Date  \
0          E001  Amit Sharma           2  25.0       1    Remote  15-01-2021
1          E002   Riya Patel           1  30.0       0    Hybrid  15-02-2020
2          E003  John Mathew           0  29.0       1    Onsite  10-03-2022
3          E004  Sneha Joshi           2  28.0       0    Remote  22-04-2021
4          E005  Arjun Verma           3  35.0       1    Hybrid  18-05-2020
..          ...          ...         ...   ...     ...       ...         ...
245        E006   Neha Singh           1  27.0       0    Remote  01-06-2023
246        E007    Rahul Das           0  40.0       1    Onsite  07-07-2019
247        E008  Pooja Mehta           3  29.0       0    Hybrid  12-08-2022
248        E009  Sameer Khan           2  31.0       1    Remote  30-09-2021
249        E010   Kiran Nair           1  26.0       0    Onsite  05-10-2023

     Hours_Worked_Per_Week  Projects_Completed  Performance_Score  \
0                     40.0                   5             4.5000
1                     38.0                   4             4.0000
2                     45.0                   6             4.3125
3                     41.0                   5             4.2000
4                     50.0                   7             4.8000
..                     ...                 ...                ...
245                   36.0                   3             3.9000
246                   48.0                   8             4.7000
247                   42.0                   6             4.3000
248                   41.0                   5             4.1000
249                   37.0                   4             4.3125

           Salary   Location  Gender_encoded  Department_encoded
0    50000.000000       Pune               1                   2
1    45000.000000     Mumbai               0                   1
2    60000.000000  Bangalore               1                   0
3    52000.000000      Delhi               0                   2
4    50666.666667       Pune               1                   3
..            ...        ...             ...                 ...
245  42000.000000     Mumbai               0                   1
246  65000.000000      Delhi               1                   0
247  48000.000000  Bangalore               0                   3
248  51000.000000       Pune               1                   2
249  43000.000000     Mumbai               0                   1
```

```
[250 rows x 14 columns]
```

Q3. One-Hot Encoding

Apply One-Hot Encoding on:

Work_Mode,Location

Display the dataset and check how many new columns are created

```
onehot_df = pd.get_dummies(df, columns=['Work_Mode', 'Location'])

onehot_df.head()
```

	Employee_ID	Name	Department	Age	Gender	Join_Date	Hours_Worked_Per_Week	Projects_Completed	Performance_Score	
0	E001	Amit Sharma	2	25.0	1	15-01-2021	40.0	5	4.5000	5(
1	E002	Riya Patel	1	30.0	0	15-02-2020	38.0	4	4.0000	4!
2	E003	John Mathew	0	29.0	1	10-03-2022	45.0	6	4.3125	6(
3	E004	Sneha Joshi	2	28.0	0	22-04-2021	41.0	5	4.2000	5!
4	E005	Arjun Verma	3	35.0	1	18-05-2020	50.0	7	4.8000	5(

Q4. Normalization (Min-Max Scaling)

Normalize the following columns:

Salary,Hours_Worked_Per_Week

Use MinMaxScaler and display results

```
scaler = MinMaxScaler()

df['Salary_nor']= scaler.fit_transform(df[['Salary']])
df[',Hours_Worked_Per_Week']= scaler.fit_transform(df[['Hours_Worked_Per_Week']])

print(df)
```

```
    Employee_ID         Name  Department   Age  Gender Work_Mode   Join_Date  \
0          E001  Amit Sharma           2  25.0       1    Remote  15-01-2021
1          E002   Riya Patel           1  30.0       0    Hybrid  15-02-2020
2          E003  John Mathew           0  29.0       1    Onsite  10-03-2022
3          E004  Sneha Joshi           2  28.0       0    Remote  22-04-2021
4          E005  Arjun Verma           3  35.0       1    Hybrid  18-05-2020
..          ...          ...         ...   ...     ...       ...         ...
245        E006   Neha Singh           1  27.0       0    Remote  01-06-2023
246        E007    Rahul Das           0  40.0       1    Onsite  07-07-2019
247        E008  Pooja Mehta           3  29.0       0    Hybrid  12-08-2022
248        E009  Sameer Khan           2  31.0       1    Remote  30-09-2021
249        E010   Kiran Nair           1  26.0       0    Onsite  05-10-2023

     Hours_Worked_Per_Week  Projects_Completed  Performance_Score  \
0                     40.0                   5             4.5000
1                     38.0                   4             4.0000
2                     45.0                   6             4.3125
3                     41.0                   5             4.2000
4                     50.0                   7             4.8000
..                     ...                 ...                ...
245                   36.0                   3             3.9000
246                   48.0                   8             4.7000
247                   42.0                   6             4.3000
248                   41.0                   5             4.1000
249                   37.0                   4             4.3125

           Salary   Location  Gender_encoded  Department_encoded  Salary_nor  \
0    50000.000000       Pune               1                   2    0.347826
1    45000.000000     Mumbai               0                   1    0.130435
2    60000.000000  Bangalore               1                   0    0.782609
3    52000.000000      Delhi               0                   2    0.434783
4    50666.666667       Pune               1                   3    0.376812
..            ...        ...             ...                 ...         ...
245  42000.000000     Mumbai               0                   1    0.000000
246  65000.000000      Delhi               1                   0    1.000000
247  48000.000000  Bangalore               0                   3    0.260870
248  51000.000000       Pune               1                   2    0.391304
249  43000.000000     Mumbai               0                   1    0.043478
```

```
     ,Hours_Worked_Per_Week
0                  0.285714
1                  0.142857
2                  0.642857
3                  0.357143
4                  1.000000
..                      ...
245                0.000000
246                0.857143
247                0.428571
248                0.357143
249                0.071429

[250 rows x 16 columns]
```

Q5. Standardization (Scaling)

Apply StandardScaler on:

Age,Projects_Completed

Display the transformed values.

```
scaler=StandardScaler()

df['Age_std']= scaler.fit_transform(df[['Age']])
df['Projects_Completed_std']= scaler.fit_transform(df[['Projects_Completed']])

print(df)
```

```
    Employee_ID         Name  Department   Age  Gender Work_Mode   Join_Date  \
0          E001  Amit Sharma           2  25.0       1    Remote  15-01-2021
1          E002   Riya Patel           1  30.0       0    Hybrid  15-02-2020
2          E003  John Mathew           0  29.0       1    Onsite  10-03-2022
3          E004  Sneha Joshi           2  28.0       0    Remote  22-04-2021
4          E005  Arjun Verma           3  35.0       1    Hybrid  18-05-2020
..          ...          ...         ...   ...     ...       ...         ...
245        E006   Neha Singh           1  27.0       0    Remote  01-06-2023
246        E007    Rahul Das           0  40.0       1    Onsite  07-07-2019
247        E008  Pooja Mehta           3  29.0       0    Hybrid  12-08-2022
248        E009  Sameer Khan           2  31.0       1    Remote  30-09-2021
249        E010   Kiran Nair           1  26.0       0    Onsite  05-10-2023

     Hours_Worked_Per_Week  Projects_Completed  Performance_Score  \
0                     40.0                   5             4.5000
1                     38.0                   4             4.0000
2                     45.0                   6             4.3125
3                     41.0                   5             4.2000
4                     50.0                   7             4.8000
..                     ...                 ...                ...
245                   36.0                   3             3.9000
246                   48.0                   8             4.7000
247                   42.0                   6             4.3000
248                   41.0                   5             4.1000
249                   37.0                   4             4.3125

           Salary   Location  Gender_encoded  Department_encoded  Salary_nor  \
0    50000.000000       Pune               1                   2    0.347826
1    45000.000000     Mumbai               0                   1    0.130435
2    60000.000000  Bangalore               1                   0    0.782609
3    52000.000000      Delhi               0                   2    0.434783
4    50666.666667       Pune               1                   3    0.376812
..            ...        ...             ...                 ...         ...
245  42000.000000     Mumbai               0                   1    0.000000
246  65000.000000      Delhi               1                   0    1.000000
247  48000.000000  Bangalore               0                   3    0.260870
248  51000.000000       Pune               1                   2    0.391304
249  43000.000000     Mumbai               0                   1    0.043478

     ,Hours_Worked_Per_Week   Age_std  Projects_Completed_std
0                  0.285714 -1.172018               -0.211604
1                  0.142857  0.000000               -0.916949
2                  0.642857 -0.234404                0.493742
3                  0.357143 -0.468807               -0.211604
4                  1.000000  1.172018                1.199088
..                      ...       ...                     ...
245                0.000000 -0.703211               -1.622295
246                0.857143  2.344036                1.904433
247                0.428571 -0.234404                0.493742
248                0.357143  0.234404               -0.211604
249                0.071429 -0.937614               -0.916949

[250 rows x 18 columns]
```

Q6. Compare Scaling Methods Apply both:

MinMaxScaler

StandardScaler

On the Salary column.

Show both results side by side

```
minmax= MinMaxScaler()
stdscaler= StandardScaler()

df['Salary_minmax']= minmax.fit_transform(df[['Salary']])
df['Salary_stdscaler']= stdscaler.fit_transform(df[['Salary']])

print(df)
```

```
    Employee_ID         Name  Department   Age  Gender Work_Mode   Join_Date  \
0          E001  Amit Sharma           2  25.0       1    Remote  15-01-2021
1          E002   Riya Patel           1  30.0       0    Hybrid  15-02-2020
2          E003  John Mathew           0  29.0       1    Onsite  10-03-2022
3          E004  Sneha Joshi           2  28.0       0    Remote  22-04-2021
4          E005  Arjun Verma           3  35.0       1    Hybrid  18-05-2020
..          ...          ...         ...   ...     ...       ...         ...
245        E006   Neha Singh           1  27.0       0    Remote  01-06-2023
246        E007    Rahul Das           0  40.0       1    Onsite  07-07-2019
247        E008  Pooja Mehta           3  29.0       0    Hybrid  12-08-2022
248        E009  Sameer Khan           2  31.0       1    Remote  30-09-2021
249        E010   Kiran Nair           1  26.0       0    Onsite  05-10-2023

     Hours_Worked_Per_Week  Projects_Completed  Performance_Score  \
0                     40.0                   5             4.5000
1                     38.0                   4             4.0000
2                     45.0                   6             4.3125
3                     41.0                   5             4.2000
4                     50.0                   7             4.8000
..                     ...                 ...                ...
245                   36.0                   3             3.9000
246                   48.0                   8             4.7000
247                   42.0                   6             4.3000
248                   41.0                   5             4.1000
249                   37.0                   4             4.3125

           Salary   Location  Gender_encoded  Department_encoded  Salary_nor  \
0    50000.000000       Pune               1                   2    0.347826
1    45000.000000     Mumbai               0                   1    0.130435
2    60000.000000  Bangalore               1                   0    0.782609
3    52000.000000      Delhi               0                   2    0.434783
4    50666.666667       Pune               1                   3    0.376812
..            ...        ...             ...                 ...         ...
245  42000.000000     Mumbai               0                   1    0.000000
246  65000.000000      Delhi               1                   0    1.000000
247  48000.000000  Bangalore               0                   3    0.260870
248  51000.000000       Pune               1                   2    0.391304
249  43000.000000     Mumbai               0                   1    0.043478

     ,Hours_Worked_Per_Week   Age_std  Projects_Completed_std  Salary_minmax  \
0                  0.285714 -1.172018               -0.211604       0.347826
1                  0.142857  0.000000               -0.916949       0.130435
2                  0.642857 -0.234404                0.493742       0.782609
3                  0.357143 -0.468807               -0.211604       0.434783
4                  1.000000  1.172018                1.199088       0.376812
..                      ...       ...                     ...            ...
245                0.000000 -0.703211               -1.622295       0.000000
246                0.857143  2.344036                1.904433       1.000000
247                0.428571 -0.234404                0.493742       0.260870
248                0.357143  0.234404               -0.211604       0.391304
249                0.071429 -0.937614               -0.916949       0.043478

     Salary_stdscaler
0           -0.097451
1           -0.828333
2            1.364312
3            0.194902
4            0.000000
```

Q7. Build Preprocessing Pipeline Create a pipeline that:

Applies encoding to categorical columns

Applies scaling to numerical columns

Use:

ColumnTransformer ,Pipeline

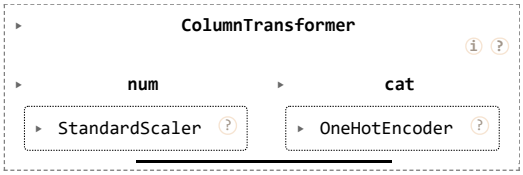
```
# handle_unknown=
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder

categorcal_col= ['Gender', 'Department', 'Work_Mode', 'Location']
numerical_col= ['Age', 'Salary', 'Hours_Worked_Per_Week', 'Performance_Score', 'Projects_Completed']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_col),
        ('cat', OneHotEncoder(drop= 'first', sparse_output=False), categorcal_col)
    ]
)

preprocessor
```



Q8. Apply Pipeline

Apply the pipeline on the dataset.

Display:

Transformed dataset

Shape of final datase

```
# STEP 1: Fill missing values
df_pipeline = df.copy()
df_pipeline['Age'].fillna(df_pipeline['Age'].median(), inplace=True)
df_pipeline['Salary'].fillna(df_pipeline['Salary'].mean(), inplace=True)
df_pipeline['Hours_Worked_Per_Week'].fillna(df_pipeline['Hours_Worked_Per_Week'].median(), inplace=True)
df_pipeline['Performance_Score'].fillna(df_pipeline['Performance_Score'].mean(), inplace=True)

# STEP 2: Select only required columns
X = df_pipeline[categorical_cols + numerical_cols]

# STEP 3: Transform
X_transformed = preprocessor.fit_transform(X)

# STEP 4: Get feature names
cat_feature_names = preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_cols)
all_feature_names = list(cat_feature_names) + numerical_cols

# STEP 5: Create dataframe
df_final = pd.DataFrame(X_transformed, columns=all_feature_names)

print(f"Final Shape: {df_final.shape}")  # ✅ (250, 14)
print("Feature Names:")
for i, col in enumerate(all_feature_names, 1):
    print(f"  {i}. {col}")
```

```
Final Shape: (250, 14)
Feature Names:
  1. Gender_1
  2. Department_1
  3. Department_2
  4. Department_3
  5. Work_Mode_Onsite
  6. Work_Mode_Remote
  7. Location_Delhi
  8. Location_Mumbai
  9. Location_Pune
  10. Age
  11. Salary
  12. Hours_Worked_Per_Week
  13. Performance_Score
  14. Projects_Completed
/tmp/ipykernel_2153/3770244921.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df_pipeline['Age'].fillna(df_pipeline['Age'].median(), inplace=True)
```

```
/tmp/ipykernel_2153/3770244921.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df_pipeline['Salary'].fillna(df_pipeline['Salary'].mean(), inplace=True)
/tmp/ipykernel_2153/3770244921.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df_pipeline['Hours_Worked_Per_Week'].fillna(df_pipeline['Hours_Worked_Per_Week'].median(), inplace=True)
/tmp/ipykernel_2153/3770244921.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df_pipeline['Performance_Score'].fillna(df_pipeline['Performance_Score'].mean(), inplace=True)
```

Q9. Conceptual Question

Why is scaling important in Python?

Answer =

1.Distance-based algorithms (KNN, K-Means, SVM) need equal scales

2.Gradient descent converges faster with scaled features

3.Prevents numerical overflow/underflow

4.Fair feature contribution regardless of original range

5.Regularization (L1/L2) works correctly only on scaled data

6.Improved model interpretability

Algorithms needing scaling: Linear Regression, Logistic Regression, SVM, Neural Networks, K-Means, KNN, PCA

Algorithms NOT needing scaling: Decision Trees, Random Forests, Gradient Boosting

Q10. Conceptual Question

Why do we convert categorical data into numerical form?

Answer

Mathematical Compatibility: Machine learning algorithms operate strictly on mathematical equations, matrix multiplications, and linear algebra operations. They cannot calculate loss, gradients, or weights directly on text strings like "Remote" or "IT".

Enables Quantitative Feature Comparison: Converting categories into numbers (e.g., binary flags in One-Hot Encoding) allows algorithms to evaluate presence/absence as mathematical variables (0 or 1).

Optimizes Model Performance: Numerical encoding enables optimization tools (like loss functions and backpropagation) to adjust parameters during model training.

```
Start coding or generate with AI.
```